

Ex.No:2 Developing and Deploying a Simple Chaincode

Aim

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Procedure Step 1: Navigate to the Test Network

Open the terminal and move to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

Start the blockchain network with Certificate Authorities.

```
./network.sh up createChannel -ca
```

Expected Output

```
Creating channel 'mychannel'<br/>Starting peer0.org1<br/>Starting  
peer0.org2<br/>Starting orderer.example.com<br/>Network Started Successfully
```

```

kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
Removing generated chaincode docker images

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ docker ps -a | grep -E 'peer0|orderer|ca_'
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ ./network.sh up createChannel -ca
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb
with crypto from 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] Running 4/4
  ✓ Network fabric_test      Created      0.1s
  ✓ Container ca_org2        Started      0.5s
  ✓ Container ca_orderer     Started      0.6s
  ✓ Container ca_org1        Started      0.7s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/
fabric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/08/20 12:20:36 [INFO] Configuration file location: /home/kali/fabric-dev/fabric-samples/test-network/organizations/peer
Organizations/org1.example.com/fabric-ca-client-config.yaml
2026/08/20 12:20:36 [INFO] TLS Enabled
2026/08/20 12:20:36 [INFO] Stored root CA certificate at /home/kali/fabric-dev/fabric-samples/test-network/organizations/pee
rOrganizations/org1.example.com/msp/cacerts/localhost-7054-ca-org1.pem
2026/08/20 12:20:36 [INFO] Stored Issuer public key at /home/kali/fabric-dev/fabric-samples/test-network/organizations/peer0
rganizations/org1.example.com/msp/IssuerPublicKey
2026/08/20 12:20:36 [INFO] Stored Issuer revocation public key at /home/kali/fabric-dev/fabric-samples/test-network/organiza
tions/peer0Organizations/org1.example.com/msp/IssuerRevocationPublicKey
+ res=0
Creating Org1 Identities
Enrolling the CA admin
+ fabric-ca-client enroll -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/kali/fabric-dev/fab
ric-samples/test-network/organizations/fabric-ca/org1/ca-cert.pem
2026/08/20 12:20:36 [INFO] Created a default configuration file at /home/kali/fabric-dev/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/fabric-ca-client-config.yaml
2026/08/20 12:20:36 [INFO] TLS Enabled
2026/08/20 12:20:36 [INFO] generating key: 6{A:ecdsa S:256}
2026/08/20 12:20:36 [INFO] encoded CSR

```

Figure 2.1: Fabric network and channel creation

Step 3: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic chaincode.

```

./network.sh deployCC \<br/>-ccl javascript \<br/>-ccn basic \<br/>-ccp
../asset-transfer-basic/chaincode-javascript

```

Expected Output

Packaging chaincode
Installing chaincode
Approving chaincode
Committing chaincode
Chaincode committed successfully

Figure 2.2: Chaincode deployment

peer lifecycle chaincode queryinstalled

Package ID:
basic 1.0
Version:1.0

```

kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
/home/kali/fabric-dev/fabric-samples/test-network-k8s/config/org1/core.yaml
/home/kali/fabric-dev/fabric-samples/config/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/addOrg3/compose/podman/peerconfig/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/addOrg3/compose/docker/peerconfig/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/compose/podman/peerconfig/core.yaml
/home/kali/fabric-dev/fabric-samples/test-network/compose/docker/peerconfig/core.yaml

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_ADDRESS=localhost:7051
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ echo $FABRIC_CFG_PATH
echo $CORE_PEER_LOCALMSPID
echo $CORE_PEER_ADDRESS
/home/kali/fabric-dev/fabric-samples/config
Org1MSP
localhost:7051

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629, Label: basic_1.0

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode querycommitted \
--channelID mychannel \
--name basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function": "GetAllAssets", "Args": []}'
[]

(kali@kali)~[~/fabric-dev/fabric-samples/test-network]

```

Figure 2.3: Verification of installed and committed chaincode

Step 5: Invoke the Chaincode

Initialize the ledger.

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride
orderer.example.com \
--tls \
-cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "InitLedger", "Args": []}'
```

Expected Output

Chaincode invoke successful
Transaction committed

Step 6: Query the Ledger

Retrieve all assets stored in the blockchain.

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
```

Expected Output

```
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 5,
  "Owner": "Tomoko",
  "AppraisedValue": 300,
  "ID": "asset2",
  "Color": "red",
  "Size": 5,
  "Owner": "Jen",
  "AppraisedValue": 350
}
```

```
<br/>"Owner":"Brad",<br/>"AppraisedValue":400<br/>}<br/>]
```

Step 7: Create a New Asset

```
peer chaincode invoke \<br/>-o localhost:7050 \<br/>--ordererTLSHostnameOverride  
orderer.example.com \<br/>--tls \<br/>--cafile $ORDERER_CA \<br/>-C mychannel  
\<br/>-n basic \<br/>-c  
'{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}'
```

Expected Output

Asset asset10 created successfully

Step 8: Query the Newly Created Asset

```
peer chaincode query \<br/>-C mychannel \<br/>-n basic \<br/>-c  
'{"Args":["ReadAsset","asset10"]}'
```

Expected Output

```
{<br/>"ID":"asset10",<br/>"Color":"Black",<br/>"Size":15,<br/>"Owner":"Arun",<br/>"Ap  
praisedValue":800<br/>}
```

Step 9: Transfer Asset Ownership

```
peer chaincode invoke \<br/>-o localhost:7050 \<br/>--ordererTLSHostnameOverride  
orderer.example.com \<br/>--tls \<br/>--cafile $ORDERER_CA \<br/>-C mychannel  
\<br/>-n basic \<br/>-c '{"function":"TransferAsset","Args":["asset10","David"]}'
```

Expected Output

Ownership transferred successfully

Step 10: Verify Updated Asset

```
peer chaincode query \<br/>-C mychannel \<br/>-n basic \<br/>-c  
'{"Args":["ReadAsset","asset10"]}'
```

Expected Output

```
{<br/>"ID":"asset10",<br/>"Owner":"David"<br/>}
```



```

kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
ca.org2.example.com-cert.pem \
-c '{"function":"CreateAsset","Args":["asset10","blue","5","Tom","300"]}'
2026-08-20 12:52:08.491 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:2
00 payload:{"ID":"asset10","Color":"blue","Size":5,"Owner":"Tom","AppraisedValue":300}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"Tom","Size":5}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.c
om-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/tlsca/tls
ca.org1.example.com-cert.pem \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tls
ca.org2.example.com-cert.pem \
-c '{"function":"TransferAsset","Args":["asset10","David"]}'
2026-08-20 12:52:54.660 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:2
00 payload:"Tom"

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"ReadAsset","Args":["asset10"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"David","Size":5}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset10","Owner":"David","Size":5}]

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$

```

Figure 2.4: Asset creation, transfer and final ledger verification

Step 11: Stop the Network

./network.sh down

Flow of Chaincode Deployment

Start Fabric Network
 Create Channel
 Package Chaincode
 Install Chaincode
 Approve Chaincode
 Commit Chaincode
 Invoke Chaincode
 Query Ledger
 Create Asset
 Transfer Asset
 Stop Network

Sample Output

Fabric Network Started
 Channel Created Successfully
 Chaincode Installed Successfully
 Chaincode Approved
 Chaincode Committed
 Ledger Initialized
 Asset Created Successfully
 Asset Ownership Transferred
 Ledger Updated Successfully

Result

The simple JavaScript chaincode was successfully developed, deployed, installed, approved, committed, and executed on the Hyperledger Fabric test network. Asset records were created, queried, and transferred successfully, demonstrating the basic lifecycle of smart contracts in Hyperledger Fabric.